

```

#include <iostream>
using namespace std;

class Student
{
private:
    static int count;
    int roll_no;
    string name,gender;
public:
    //Create a `register_student` method
    //which will accept name and gender from the user
    //print on screen: "Student successfully registered with a roll number
    auto-generated."
    register_student(string name,string gender)
    {
        count++;
        this->name=name;
        this->gender=gender;
        this->roll_no=count;
        cout<<"\nSuccessfully registered. Remember your roll number is "<<roll_no;
    }
    void display_student()
    {

cout<<"\nRollno:"<<roll_no<<"\tName:"<<name<<"\tGender:"<<gender;
    }
//get_count():Will return total number of students created so far
int get_count()
{
    return(count);
}
//get_roll():Return private data, that is, roll number, to outside the main method.
int get_roll()
{
    return(roll_no);
}
};

int Student::count;

int main()
{
    Student s;
    s.register_student("champak","male");
    s.display_student();
    cout<<"\nonly roll no:"<<s.get_roll();

```

```
    cout<<"\nTotal count:"<<s.get_count();  
    return 0;  
}
```

//to correct

#include <iostream>

using namespace std;

class Student

{

private:

static int count;

int roll_no;

string name,gender;

public:

//Create a `register\_student` method

**//which will accept name and gender from the
user**

**//print on screen: "Student successfully
registered with a roll number auto-generated."**

void register_student(string name,string gender)

{

count++;

this->name=name;

this->gender=gender;

this->roll_no=count;

**cout<<"\nSuccessfully registered. Remember
your roll number is "<<roll_no;**

}

```
void display_student()
{

cout<<"\nRollno:"<<roll_no<<"\tName:"<<name<<"\t
Gender:"<<gender;
}
//get_count():Will return total number of students
created so far
static int get_count()
{
    return(count);
}
//get_roll():Return private data, that is, roll number,
to outside the main method.
int get_roll()
{
    return(roll_no);
}
};
```

```
int Student::count;
```

```
int main()
{
    Student s[100];
    for (int i=0;i<3;i++)
    {
        cout<<"\nYour name:";
```

```
        string name;
        getline(cin,name);
        cout<<"\nYour gender:";
        string gender;
        cin>>gender;
        cin.ignore();
        s[i].register_student(name,gender);
    }
    cout<<"\nTotal Students Registered till
now:"<<Student::get_count();
    cout<<"\nList is:\n";
    for (int i=0;i<3;i++)
    {
        s[i].display_student();
    }
    int r_number;
    cout<<"\nEnter number to search:";
    cin>>r_number;
    bool found=false;
    for(int i=0;i<Student::get_count();i++)
    {
        if(r_number==s[i].get_roll())
        {
            cout<<"\nRecord found\n";
            s[i].display_student();
            found=true;
            break;
        }
    }
}
```

```
        if(found==false)
            cout<<"\nNot found";
    return 0;
}
```

Inheritance:

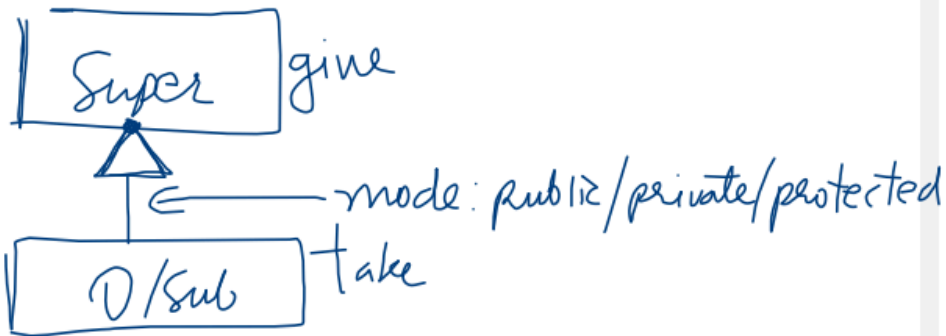
Inheritance is a property of one class taking properties from a pre-existing class.(We can take data members as well as function members.)

use:

Inheritance is faster development(Take what already exists and try to improve.), standardization(And data will exist throughout the inheritance.), and enhancement(Override certain properties and change them with reference to need and time.).

example:software:When we create a new version of software, we take certain properties and functionality from the old and add a new one.
Whatsapp

C++ inheritance happens in three modes and that reflects on the inherited data.



```
class A
{
```

```
};
```

syntax: class <sub_class_name>:<mode> <super class's name>

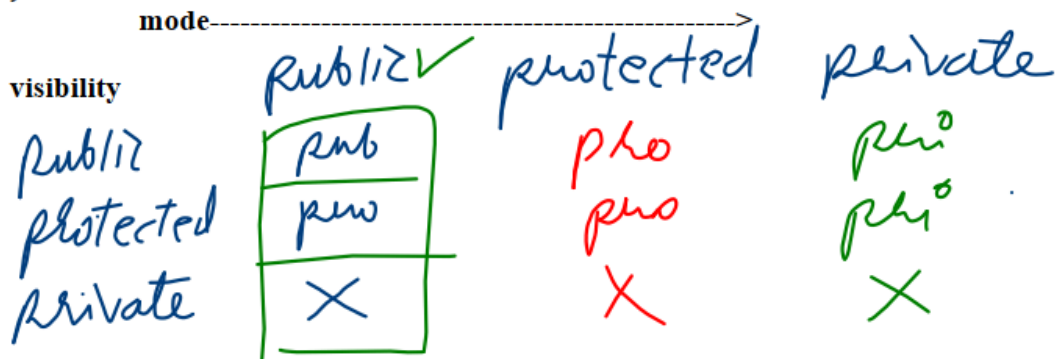
```
{
```

```
}
```

```
class B:public A
```

```
{
```

```
}
```



```
#include <iostream>
using namespace std;
```

```
class Human
{
public:
    string name;
    int age;
```

```
void displayHuman()
{
    cout << "Name: " << name << endl;
    cout << "Age: " << age << endl;
}
};

class Student : public Human
{
public:
    int rollNo;

    void displayStudent()
    {
        cout << "Roll No: " << rollNo << endl;
    }
};

int main()
{
    Student s;

    s.name = "Rahul";
    s.age = 20;
    s.rollNo = 101;

    s.displayHuman();
    s.displayStudent();

    return 0;
}
```

```
//private
#include <iostream>
using namespace std;
```

```
class Human
{
private:
```

```
    string name;
    int age;
public:
    void setHuman(string name,int age)
    {
        this->name=name;
        this->age=age;
    }
    void displayHuman()
    {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
    }
};
```

```
class Student : public Human
{
private:
    int rollNo;
public:
    void setStudent(int rollNo)
    {
        this->rollNo=rollNo;
    }
    void displayStudent()
    {
        cout << "Roll No: " << rollNo << endl;
    }
};
```

```
int main()
{
    Student s;
    /*Setting of private data is not allowed because private is not inherited.
    s.name = "Rahul";
    s.age = 20;
    */
    s.setHuman("rahul",20);//One can still inherit a public method which
    has access to private data.
    s.setStudent(101);
```

```
    s.displayHuman();  
    s.displayStudent();  
  
    return 0;  
}
```

//Nested method call from child to parent

```
#include <iostream>
```

```
using namespace std;
```

```
class Human
```

```
{
```

```
private:
```

```
    string name;
```

```
    int age;
```

```
public:
```

```
    void setHuman(string name,int age)
```

```
    {
```

```
        this->name=name;
```

```
        this->age=age;
```

```
    }
```

```
    void displayHuman()
```

```
    {
```

```
        cout << "Name: " << name << endl;
```

```
        cout << "Age: " << age << endl;
```

```
    }
```

```
};
```

```
class Student : public Human
```

```
{
```

```
private:
```

```
    int rollNo;
```

```
public:
```

```
void setStudent(string name,int age,int rollNo)//Child takes all the data  
and passes it to the parent.
```

```
{
```

```
    setHuman(name,age);//Nesting of function call from child to
```

```
parent
```

```

        this->rollNo=rollNo;
    }
    void displayStudent()
    {
        displayHuman();
        cout << "Roll No: " << rollNo << endl;
    }
};

int main()
{
    Student s;
    /*Setting of private data is not allowed because private is not inherited.
    s.name = "Rahul";
    s.age = 20;
    */
    //One can still inherit a public method which has access to private
    data.
    s.setStudent("rahul",20,101);
    s.displayStudent();

    return 0;
}

```

//Superclass constructor in inheritance

```
#include <iostream>
```

```
using namespace std;
```

```
/*
```

When an object of a subclass or derived class is created, by default it calls the object or constructor of the superclass.

```
*/
```

```
class Human
```

```
{
```

```
    public:
```

```
    Human()
```

```
    {
```

```
        cout<<"\nHuman constructor called";
```

```
    }
```

```
};
```

```
class Student : public Human
```

```
{
```

```
    public:
```

```
    Student()
```

```
    {
```

```
        cout<<"\nStudent constructor called";
```

```
    }
```

```
};
```

```
int main()
```

```
{
```

```
    Student s;
```

```
    return 0;
```

```
}
```

```
#include <iostream>
```

```
using namespace std;
```

```
/*
```

When an object of a subclass or derived class is created, by default it calls the object or constructor of the superclass.

```
*/
```

```
class Human
```

```
{
```

```
    private:
```

```
        string name;
```

```
    public:
```

```
    Human(string name)
```

```
    {
```

```
        cout<<"\nHuman constructor called";
```

```
        this->name=name;
```

```
        cout<<"\nFrom Human:name-----"<<name;
```

```
    }
```

```
};
```

```
class Student : public Human
```

```
{
```

```
private:
```

```
int rollNo;
```

```
public:
```

Student(string name,int rollNo):Human(name)//The student constructor takes data for itself as well as for the superclass `Human` and then, using the `Human` method/constructor, passes this to the superclass.

```
{
```

```
cout<<"\nStudent constructor called";
```

```
this->rollNo=rollNo;
```

```
cout<<"\nFrom Student:rollNo-->"<<rollNo;
```

```
}
```

```
};
```

```
int main()
```

```
{
```

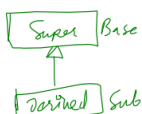
```
Student s("rahul",110);
```

```
return 0;
```

```
}
```

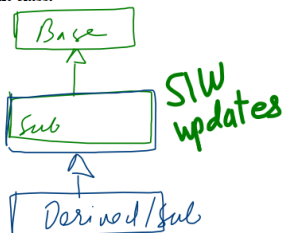
Types:

1:simple/single level



2.Multilevel:

In this a superclass creates a derived class. Later the derived class will act as a superclass for the next class.

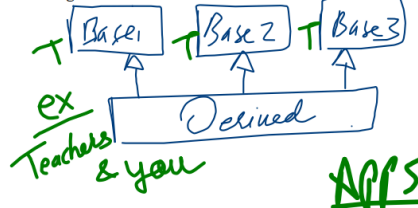


3.Hierarchical inheritance



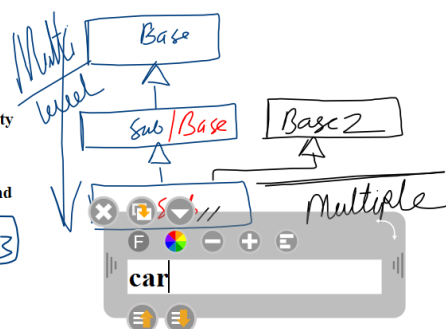
A single superclass or a base class will give its property to multiple derived or subclasses.

4.Mutiple inheritance: This allows a derived class to have multiple superclasses, taking their properties and creating one.



5.Hybrid:

This is a combination of multi-level and multiple.



```
//multilevel
#include <iostream>
using namespace std;

class Person
{
public:
    string name;

    void showName()
    {
        cout << "Name: " << name << endl;
    }
};

class Student : public Person
{
public:
    int rollNo;

    void showRollNo()
    {
        cout << "Roll No: " << rollNo << endl;
    }
};

class EngineeringStudent : public Student
{
public:
    string branch;

    void showBranch()
    {
        cout << "Branch: " << branch << endl;
    }
};

int main()
{
    EngineeringStudent s;

    s.name = "Amit";
    s.rollNo = 101;
    s.branch = "Computer Engineering";

    s.showName();
    s.showRollNo();
}
```

```

        s.showBranch();

        return 0;
    }
//hierarchical
#include <iostream>
using namespace std;

class Person
{
public:
    string name;

    void showName()
    {
        cout << "Name: " << name << endl;
    }
};

class Student : public Person
{
public:
    void study()
    {
        cout << "Student is studying." << endl;
    }
};

class Teacher : public Person
{
public:
    void teach()
    {
        cout << "Teacher is teaching." << endl;
    }
};

class Employee : public Person
{
public:
    void work()
    {
        cout << "Employee is working." << endl;
    }
};

int main()
{

```

```

    Student s;
    s.name = "Rahul";
    s.showName();
    s.study();

    Teacher t;
    t.name = "Amar";
    t.showName();
    t.teach();

    Employee e;
    e.name = "Raj";
    e.showName();
    e.work();

    return 0;
}

//multiple
#include <iostream>
using namespace std;

class Teacher
{
public:
    void teach()
    {
        cout << "Teacher can teach." << endl;
    }
};

class Researcher
{
public:
    void research()
    {
        cout << "Researcher can do research." << endl;
    }
};

class Professor : public Teacher, public Researcher
{
public:
    void conductClass()
    {
        cout << "Professor conducts class." << endl;
    }
};

```

```

int main()
{
    Professor p;

    p.teach();      // From Teacher
    p.research();   // From Researcher
    p.conductClass(); // Own function

    return 0;
}
//hybrid
#include <iostream>
using namespace std;

class Person
{
public:
    string name;

    void showName()
    {
        cout << "Name: " << name << endl;
    }
};

class Student : public Person
{
public:
    void study()
    {
        cout << "Student is studying." << endl;
    }
};

class Employee
{
public:
    void work()
    {
        cout << "Employee is working." << endl;
    }
};

class WorkingStudent : public Student, public Employee
{
public:
    void attendOffice()

```

```

    {
        cout << "Working student attends office." << endl;
    }
};

```

```

int main()
{
    WorkingStudent ws;

    ws.name = "Rahul";

    ws.showName();    // From Person → Student
    ws.study();       // From Student
    ws.work();        // From Employee
    ws.attendOffice(); // Own function

    return 0;
}

```

```

//Ambiguity resolution.
#include <iostream>
using namespace std;

```

```

class Father
{
public:
    void life()
    {
        cout << "Father teaches life." << endl;
    }
};

```

```

class Mother
{
public:
    void love()
    {
        cout << "Mother teaches love." << endl;
    }
};

```

```

class Child : public Father, public Mother
{
};

```

```

int main()
{

```

```

    Child c;

    c.life(); // Father
    c.love(); // Mother

    return 0;
}

```

```

#include <iostream>
using namespace std;
/*

```

What is ambiguity? It is Confusion of a derived or subclass where both the parents are giving an exact method to inherit and we are unclear whose method to inherit

The way C++ resolves ambiguity is
Subclass or the child class will write its own method and, within the method, would decide whose method from the parent it has to inherit.

In C++ we can also override, that is, write our own new method, which will reject the parent's method.

In C++ we can also activate both parents' methods by precisely writing them within a newly created method.

In C++ we can also decide which method I want to take, only writing that method within a newly created

```

*/
class Father
{
public:
    void life()
    {
        cout << "Father teaches life." << endl;
    }

    void eat()
    {
        cout << "Father eats non-veg." << endl;
    }
};

```

```

class Mother
{
public:
    void love()
    {
        cout << "Mother teaches love." << endl;
    }
}

```

```

    void eat()
    {
        cout << "Mother eats veg only." << endl;
    }
};

class Child : public Father, public Mother
{
    public:
        void eat()
        {
            Father::eat();//Resolving ambiguity by deciding to take property only
from the father
            Mother::eat();
            //cout<<"Child: eats chines.....everything";
        }
};

int main()
{
    Child c;

    c.life(); // No ambiguity
    c.love(); // No ambiguity

    c.eat(); // AMBIGUITY!--resolved by your call

    return 0;
}

```

//The classic diamond problem: combination of hierarchical inheritance and multiple inheritance, causing multiple activation calls to the superclass or grandparent class

```
#include <iostream>
```

```
using namespace std;
```

```

class Human
{
public:
    string name;

    Human()
    {
        cout << "Human constructor called" << endl;
    }
};

```

```
class Student : public Human
```

```

{
public:
    string degree;

    Student()
    {
        cout << "Student constructor called" << endl;
    }
};

class Employee : public Human
{
public:
    string company;

    Employee()
    {
        cout << "Employee constructor called" << endl;
    }
};

class WorkingStudent : public Student, public Employee
{
public:
    void display()
    {
        cout<<"\nWorking Student:my display";
    }
};

int main()
{
    WorkingStudent obj;
    obj.display();

    return 0;
}

```

//The solution to the problem is a virtual base class where, during inheritance, we use the keyword `virtual` before or after the `public` keyword.

```

#include <iostream>
using namespace std;

```

```

class Human
{
public:
    string name;

```

```

    Human()
    {
        cout << "Human constructor called" << endl;
    }
};

class Student : virtual public Human
{
public:
    string degree;

    Student()
    {
        cout << "Student constructor called" << endl;
    }
};

class Employee : public virtual Human
{
public:
    string company;

    Employee()
    {
        cout << "Employee constructor called" << endl;
    }
};

class WorkingStudent : public Student, public Employee
{
public:
    void display()
    {
        cout<<"\nWorking Student:my display";
    }
};

int main()
{
    WorkingStudent obj;
    obj.display();

    return 0;
}

```

```
#include <iostream>
using namespace std;
/*
```

Override is a process where a child class recodes a parent's method due to adaptability or enhancement and in the process rejects the parent's method by recoding its own new method.

```
*/
class Parent
{
public:
    void eat()
    {
        cout<<"\nParent:basic bhaji chapatii ";
    }

    void speak()
    {
        cout<<"\nParent:native language";
    }
};
class Child:public Parent
{
public:
    void eat()
    {
        cout<<"\nChild:wada pav/samosa pav ";
    }
};
int main()
{
    Child c;
    c.eat();
    c.speak();
    return 0;
}
```

```

#include <iostream>
using namespace std;
/*
pointer of type parent can ref to child also due to inheritance
but rejects override.
*/
class Parent
{
public:
    void eat()
    {
        cout<<"\nParent:basic bhaji chapatii ";
    }

    void speak()
    {
        cout<<"\nParent:native language";
    }
    virtual void fun()=0;
};
class Child:public Parent
{
public:
    void eat()
    {
        cout<<"\nChild:wada pav/samosa pav ";
    }
};
int main()
{
    Parent obj_p;
    Child obj_c;
    Parent *p=&obj_p;
    p->eat();//(*p).XXXX

    return 0;
}

```

//Using the `virtual` keyword, a pointer can refer to the current object's method only. Later `virtual` can be used to override and allow pointer-based access.

```
#include <iostream>
```

```
using namespace std;
```

```
/*
```

pointer of type parent can ref to child also due to inheritance
but rejects override.

```
*/
```

```
class Parent
```

```
{
```

```
public:
```

```
    virtual void eat()
```

```
    {
```

```
        cout<<"\nParent:basic bhaji chapatii ";
```

```
    }
```

```
    void speak()
```

```
    {
```

```
        cout<<"\nParent:native language";
```

```
    }
```

```
};
```

```
class Child:public Parent
```

```
{
```

```
    public:
```

```
    void eat()
```

```
    {
```

```
        cout<<"\nChild:wada pav/samosa pav ";
```

```
    }
```

```
};
```

```
int main()
```

```
{
```

```
    Parent obj_p;
```

```
    Child obj_c;
```

```
    Parent *p=&obj_p;
```

```
    p->eat();//This will call the parent's own method.
```

```
    p=&obj_c;
```

```
    p->eat();//Virtual function. Now this will call the child's own method.
```

```
    return 0;
```

```
}
```

/*pure virtual function

In this the base class will not carry any code but just a compulsion that any class inheriting this must implement this function.

A class which has one or more virtual methods is also called an abstract class.

This class is used just as a framework so that somebody can follow the framework and implement it in their own way.

*/

```
#include <iostream>
```

```
using namespace std;
```

```
class Parent
```

```
{
```

```
public:
```

```
    virtual void fun() = 0; // Pure virtual function
```

```
};
```

```
class Child1 : public Parent
```

```
{
```

```
public:
```

```
    void fun() override
```

```
    {
```

```
        cout << "\nChild is having fun:online social media";
```

```
    }
```

```
};
```

```
class Child2 : public Parent
```

```
{
```

```
public:
```

```
    void fun() override
```

```
    {
```

```
        cout << "\nChild is having fun:coding";
```

```
    }
```

```
};
```

```
int main()
```

```
{
```

```
    Child1 obj1;
```

```
    Child2 obj2;
```

```
    Parent *p = &obj1;
```

```
    p->fun();
```

```
    p = &obj2;
```

```
    p->fun();
```

```
    return 0;
```

```
}
```

What is object-oriented programming and what are the four pillars? Explain in your words.

Object-oriented programming is a bottom-to-top thinking process where we first think about the output needed (the objective), then we code the method (that is, the process which will get that output). During the coding of the method we will clearly understand the inputs needed for it.

Operate on four major pillars:

encapsulation: Process of combining data along with an operation on that data in a single unit (Encapsulate.)

+ve: Makes operation easy because you are combining it as a single unit.

abstraction:

Process of hiding complex implementation and giving a simple view. This improves user operation as it makes it simple to access. Done by making data private and functions public. User can access, set, or use data just by calling methods. Without knowing how, in the background, they are implemented or how results are generated

+ve: Simple view allows better usability.

inheritance:

Ability of one class to take properties from pre-existing

+ve: Faster development, ease of operation, standardization, enhancement

polymorphism. :

Ability to exist in multiple forms. There are two types of polymorphism:

1. Overloading:(compile time)

Exists within a single class, where multiple functions would have exactly one identifier but different functionality, done so that new functionality and support for a new data type can be added

2. Overriding(runtime)

Exists within inheritance only, where the child will re-code the parent's method in terms of adaptability or need, but in the process rejecting the parent's inheritance by coding its own method.

**Error:kind of state
onwards which
normal execution
is not possible may
be caused by
wrong
syntax,wrong**

input, a logical error, or hardware failure.

Exception: Reporting mechanism of error

Exception Handling: Process of either having an alternate flow by handling the error, or trying to continue normal flow even if an error exists, or simply closing the system in a safe manner

There are three blocks to handle:

- try
- Block that part of the code which may cause an exception.
catch
- Keeping that part of the code which will report the type of exception and then either lead to a safe close or an alternate
throw
Detecting a particular state that will cause an error, we will throw an exception.

```
#include <iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int no1, no2;
```

```
    cout << "Enter two numbers: ";
```

```
    cin >> no1 >> no2;
```

```
    try
```

```
    {
```

```
        if (no2 == 0)
```

```
        {
```

```
            throw no2;
```

```
        }
```

```
        int answer = no1 / no2;
```

```
        cout << "Answer = " << answer << endl;
```

```
    }
```

```

    catch (int value)
    {
        cout << "Error: Division by zero is not allowed. AS NUMBER 2 IS " << value<<endl;
    }

    cout << "Program continues normally..." << endl;

    return 0;
}

```

1. Risky code goes inside try

```
try
```

```
{
    ...
}
```

2. Exception is generated using throw

```
throw no2;
```

3. Matching catch handles it

```
catch(int value)
```

Because:

```
throw no2;
```

throws an int.

Therefore:

```
catch(int value)
```

matches it.

Most important rule

```
throw datatype
```

↓

matching catch datatype

For example:

```
throw 10;
```

can be caught by:

```
catch(int x)
```

```
#include <iostream>
using namespace std;
/*
```

Class and their relationship

A class can be in IS-A (Inheritance)relationship or HAS-A(ContainerShip) relationship.

IS-A:A base class is a superclass of a derived class.

A child is inheriting from a parent. It exists in an "is-a" relationship.

HAS-A:Also known as containership, where one class is part of another class. Generally used to define complex entities or an entity which is a combination of multiple small entities
example:Car has an engine.

```
*/
class Engine
{
    public:
        void start()
        {
            cout<<"BHROOOOOM";
        }
};
class Car
{
    public:

    Engine e;
    void start_car()
    {
        e.start();
        cout<<"\nCar started.....";
    }

};
int main()
{
    Car obj;
    obj.start_car();

    return 0;
}
```

